

瞭解 Skaffold

來源：<https://codelabs.developers.google.com/understanding-skaffold?hl=zh-tw>

步驟數：8

目錄

1. [1. 目標](#)
2. [2. 事前準備](#)
3. [3. 開始使用 Skaffold](#)
4. [4. 建構](#)
5. [5. 部署](#)
6. [6. 設定檔](#)
7. [7. 使用 Skaffold 進行開發](#)
8. [8. 恭喜！](#)

1. 目標

Scaffold 是一種工具，可處理建構、推送及部署應用程式的工作流程。您可以使用 Scaffold 輕鬆設定本機開發工作區、簡化內部開發迴圈，並與 **Kustomize** 和 **Helm** 等其他工具整合，協助管理 Kubernetes 資訊清單。

在本教學課程中，您將瞭解 Scaffold 的一些核心概念，並使用 Scaffold 自動執行內部開發迴圈，然後部署應用程式。

您將學會以下內容：

- 設定並啟用 Scaffold，在本機開發
 - 建構及執行簡單的 Go 應用程式
 - 使用 Scaffold 管理本機應用程式部署作業
 - 算繪資訊清單並部署應用程式
-

步驟 2 · 約 0 分鐘

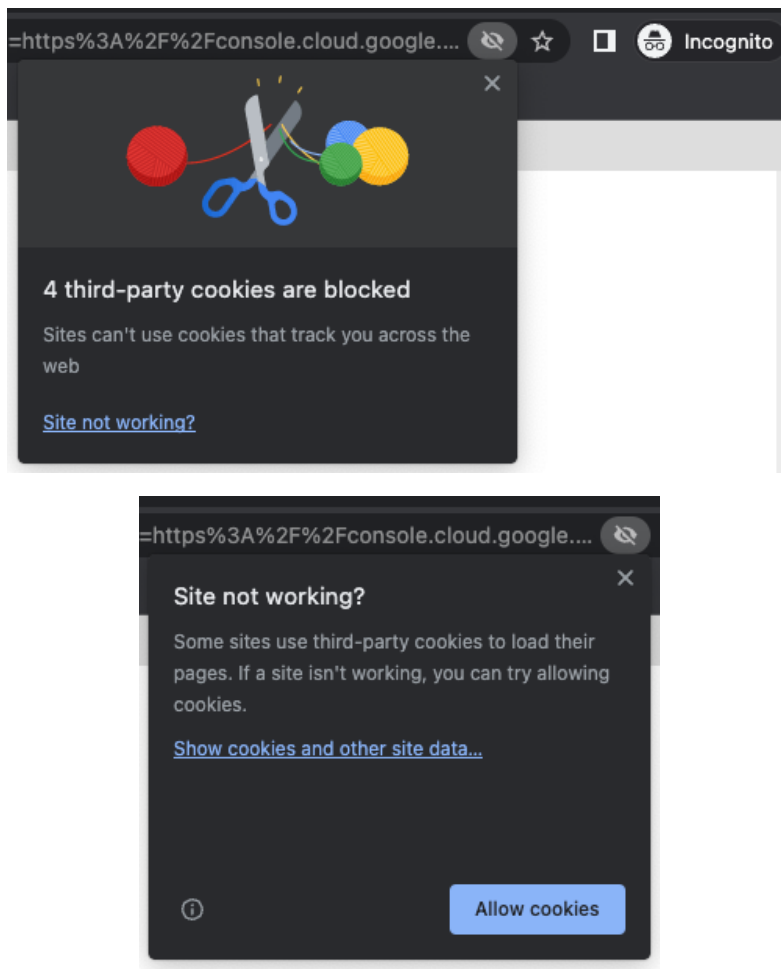
2. 事前準備

準備工作區

1. 前往下列網址開啟 Cloud Shell 編輯器：

```
https://shell.cloud.google.com
```

允許第三方 Cookie。按一下「網站無法正常運作」，然後按一下「允許 Cookie」。



2. 如果尚未執行，請在終端機視窗中，使用下列指令複製應用程式來源：

```
git clone https://github.com/GoogleCloudPlatform/software-delivery-workshop.git
```

3. 變更至複製的存放區目錄：

```
cd software-delivery-workshop/labs/understanding-skaffold/getting-started
```

4. 執行下列指令，將 Cloud Shell 工作區設為目前目錄：



```
cloudshell workspace .
```

準備專案

1. 執行下列指令，確認 Google Cloud 雲端專案設定正確：



```
gcloud config set project {{project-id}}
```

3. 開始使用 Skaffold

1. 執行下列指令，建立頂層 Skaffold 設定檔 `skaffold.yaml`：



```
cat <<EOF > skaffold.yaml
apiVersion: skaffold/v2beta21
kind: Config
metadata:
  name: getting-started-kustomize
build:
  tagPolicy:
    gitCommit:
      ignoreChanges: true
  artifacts:
  - image: skaffold-kustomize
    context: app
    docker:
      dockerfile: Dockerfile
deploy:
  kustomize:
    paths:
    - overlays/dev
profiles:
- name: staging
  deploy:
    kustomize:
      paths:
      - overlays/staging
- name: prod
  deploy:
    kustomize:
      paths:
      - overlays/prod
EOF
```

2. 在 IDE 窗格中開啟 `skaffold.yaml` 檔案。這是定義 Skaffold 管道的頂層設定檔。

請注意類似 Kubernetes 的 YAML 格式，以及 YAML 中的下列區段：

- `build`
- `deploy`
- `profiles`

這些區段定義應用程式的建構和部署方式，以及每個部署目標的設定檔。

如要進一步瞭解 Skaffold 階段的完整清單，請參閱 Skaffold Pipeline Stages [說明文件](#)。

步驟 4 · 約 0 分鐘

4. 建構

`build` 區段包含定義應用程式建構方式的設定。在本例中，您可以看到 `git` 標記的處理方式設定，以及定義應用程式所含容器映像檔的 `artifacts` 區段。

此外，您可以在本節中查看用於建構圖片的 `Dockerfile` 參照。Skaffold 也支援其他建構工具，例如

`Jib`、`Maven`、`Gradle`、雲端原生 `Buildpacks`、`Bazel` 和自訂指令碼。如要進一步瞭解這項設定，請參閱 [Skaffold 建構說明文件](#)。

步驟 5 · 約 0 分鐘

5. 部署

`deploy` 區段包含定義應用程式部署方式的設定。在本例中，您可以看到預設部署作業的範例，該部署作業會設定 Skaffold 使用 `Kustomize` 工具。

`Kustomize` 工具可將一組常見的元件 YAML 檔案 (位於 `base` 目錄下) 與一或多個「疊加層」合併，產生 Kubernetes 資訊清單，這些疊加層通常對應一或多個部署目標，例如 *dev*、*test*、*staging* 和 *production* 或類似項目。

在這個範例中，您可以看到三個目標 (*dev*、*staging* 和 *prod*) 的兩個疊加層。*dev* 疊加層會在本地開發期間使用，*staging* 和 *prod* 疊加層則會在透過 Skaffold 部署時使用。

6. 設定檔

`profiles` 區段包含設定，可定義不同環境的建構、測試和部署設定。不同環境通常是應用程式部署管道中的不同環境，例如本例中的 `staging` 或 `prod`。也就是說，您可輕鬆管理內容需因目標環境而異的資訊清單，不必重複設定樣板。

`profiles` 區段中的設定可以取代或修補主要設定中的任何項目 (例如 `build`、`test` 或 `deploy` 區段)。

舉例來說，請開啟 `overlays > prod > deployment.yaml` 檔案。請注意，應用程式的副本數量在這裡設定為三個，覆寫了基本設定。

瀏覽應用程式原始碼。

1. 在 IDE 窗格中開啟下列檔案：`app > main.go`。這是一個簡單的 Golang 應用程式，每秒會將字串寫入 `stdout`。
2. 請注意，應用程式也會輸出執行所在 Kubernetes Pod 的名稱。

查看 Dockerfile

1. 在 IDE 窗格中開啟 `app > Dockerfile` 檔案。這個檔案包含一連串指令，可為 `main.go` 檔案建構應用程式容器映像檔，並在頂層 `skaffold.yaml` 檔案中參照。
-

步驟 7 · 約 0 分鐘

7. 使用 Skaffold 進行開發

設定 Kubernetes 環境

1. 執行下列指令，確認本機 Kubernetes 叢集正在執行並已完成設定：

```
minikube start
```

這可能需要幾分鐘的時間。如果叢集已順利啟動，您應該會看到下列輸出內容：

```
Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

2. 執行下列指令，為 `dev`、`staging` 和 `prod` 建立 Kubernetes 命名空間：

```
kubectl apply -f namespaces.yaml
```

您應該會看到以下的輸出內容：

```
namespace/dev created
namespace/staging created
namespace/prod created
```

使用 Skaffold 進行本機開發

1. 執行下列指令，建構應用程式並部署至 Cloud Shell 中執行的本機 Kubernetes 叢集：

```
skaffold dev
```

您應該會看到應用程式容器建構程序正在執行 (可能需要一分鐘)，然後應用程式輸出內容每秒重複一次：

```
[skaffold-kustomize] Hello world from pod skaffold-kustomize-dev-xxxxxxxx-xxxxx
```

請注意，確切的 Pod 名稱會與上述一般輸出內容有所不同。

變更應用程式

應用程式現在會在 Kubernetes 本機叢集中執行，您可以變更程式碼，Skaffold 會自動重建應用程式，並重新部署至叢集。

1. 在 IDE 窗格中開啟 `app > main.go` 檔案，然後變更輸出字串：

```
"Hello world from pod %s!\n"
```

改為下列程式碼：

```
"Hello Skaffold world from pod %s!\n"
```

完成變更後，您應該會看到 Skaffold 重建映像檔並重新部署至叢集，而終端機視窗中會顯示輸出內容的變更。

- 現在，在 IDE 窗格的「app > main.go」檔案中，變更以下程式碼行：

```
time.Sleep(time.Second * 1)
```

到

```
time.Sleep(time.Second * 10)
```

您應該會再次看到應用程式重建並重新部署，輸出行每 10 秒出現一次。

變更 Kubernetes 設定

接下來，您將變更 Kubernetes 設定，Skaffold 會再次自動重新部署。

- 在 IDE 中開啟 `base > deployment.yaml` 檔案，並變更下列程式碼行：

```
replicas: 1
```

到

```
replicas: 2
```

重新部署應用程式後，您應該會看到兩個正在執行的 Pod，且名稱各不相同。

- 現在，請將 `base > deployment.yaml` 檔案中的同一行程式碼改回：

```
replicas: 1
```

您應該會看到其中一個 Pod 從服務中移除，只剩下一個。

- 最後，在終端機視窗中按下 `Ctrl-C`，停止 Skaffold 本機開發。

發行版本

接著，您要建構發布映像檔，並將其部署至叢集，藉此建立版本。

- 執行下列指令來建構版本：

```
skaffold build --file-output artifacts.json
```

這個指令會建構最終映像檔 (如有必要)，並將發布詳細資料輸出至 `artifacts.json` 檔案。

如果您想使用 Cloud Deploy 等工具將內容部署到叢集，這個檔案會包含發布資訊。也就是說，在發布前，構件無法變更。

- 執行下列指令，查看 `artifacts.json` 檔案的內容：

```
cat artifacts.json | jq
```

請注意，檔案包含最終部署作業中使用的圖片參照。

部署至測試環境

1. 執行下列指令，使用 `staging` 設定檔部署版本：

```
scaffold deploy --profile staging --build-artifacts artifacts.json --tail
```

部署完成後，您應該會看到兩個 Pod 的輸出內容，如下所示：

```
[scaffold-kustomize] Hello world from pod scaffold-kustomize-staging-xxxxxxxxxx-xxxxx!
```

2. 在終端機視窗中按下 Ctrl-C，停止 Scaffold 輸出。
3. 執行下列指令，觀察應用程式在叢集中運作的情形：

```
kubectl get all --namespace staging
```

您應該會看到兩個不同的 Pod 名稱，因為應用程式的 `staging` 設定檔指定部署中應有兩個副本。

部署至正式環境

1. 現在執行下列指令，使用 `prod` 設定檔部署版本：

```
scaffold deploy --profile prod --build-artifacts artifacts.json --tail
```

部署完成後，您應該會看到來自三個 Pod 的輸出內容，類似如下：

```
[scaffold-kustomize] Hello world from pod scaffold-kustomize-prod-xxxxxxxxxx-xxxxx!
```

2. 在終端機視窗中按下 Ctrl-C，停止 Scaffold 輸出。

您應該會看到三個不同的 Pod 名稱，因為應用程式的 `prod` 設定檔指定部署作業中應有三個副本。

3. 執行下列指令，觀察應用程式在叢集中運作的情形：

```
kubectl get all --namespace prod
```

您應該會看到輸出內容，其中包含類似下列的行，顯示 `prod` 部署作業：

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/scaffold-kustomize-prod	3/3	3	3	16m

您也應該會看到三個應用程式 Pod 正在執行。

NAME	READY	STATUS	RESTARTS	AGE
pod/skaffold-kustomize-prod-xxxxxxxxxx-xxxxx	1/1	Running	0	10m
pod/skaffold-kustomize-prod-xxxxxxxxxx-xxxxx	1/1	Running	0	10m
pod/skaffold-kustomize-prod-xxxxxxxxxx-xxxxx	1/1	Running	0	10m

步驟 8 · 約 0 分鐘

8. 恭喜！

恭喜！您已完成 `Understanding Skaffold` 實驗室，並瞭解如何設定及使用 Skaffold 進行本機開發和應用程式部署。

後續步驟：

如要進一步瞭解 Skaffold，請參閱下列文章：

- [深入瞭解 Skaffold](#)

清除所用資源

1. 執行下列指令，關閉本機叢集：

```
minikube delete
```
